

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом
Вариант 4р

Студент гр. 4345

Трушкин А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм поиска с возвратом.

Постановка задачи

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.

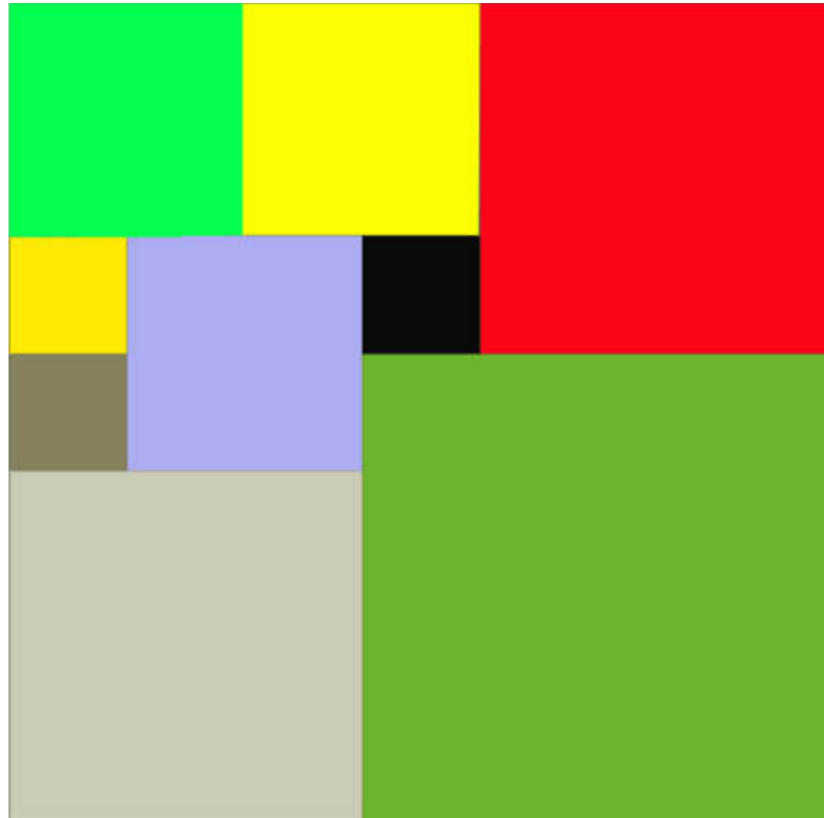


Рисунок 1 – Столешница 7×7

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N ($2 \leq N \leq 20$).

Выходные данные

Одно число K , задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу (квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых

числа x, y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 4p - Рекурсивный бэктрекинг. Расширение задачи на прямоугольные поля, рёбра квадратов меньше рёбер поля. Подсчёт количества вариантов покрытия минимальным числом квадратов.

Выполнение работы

1. Описание алгоритма.

На каждом шаге алгоритм находит первую свободную клетку прямоугольного поля $N \times M$. Для неё перебираются все возможные размеры квадратов, начиная с максимально возможного (ограниченного краями поля и уже занятыми клетками) до минимального (1x1). Если выбор приводит к заполнению всего поля, текущее количество фигур сравнивается с лучшим найденным результатом. Если решение не оптимально или ведет в тупик, алгоритм совершает возврат к предыдущему состоянию и пробует следующий размер.

2. Способ хранения частичных решений

Для хранения состояния задачи и промежуточных результатов используются:

- `board`: двумерный массив (матрица), который отмечает занятые (1) и свободные (0) клетки. Позволяет проверять доступность места для нового квадрата.
- `current_list`: динамический массив (список), хранящий кортежи (`row`, `column`, `size`) — координаты и размер текущих размещенных квадратов.
- `best_list`: динамический массив, в котором сохраняется оптимальный набор квадратов для покрытия.
- `min_squares`: целочисленный список, хранящий минимальное количество квадратов, найденное на текущий момент.
- `count_variants`: счетчик, позволяющий определить количество различных вариантов покрытия при минимальном числе квадратов.

3. Используемые оптимизации алгоритма

- Жадный выбор: Квадраты перебираются от максимально возможного размера до минимального. Это позволяет быстрее заполнить площадь поля крупными фигурами и раньше прийти к потенциально оптимальному решению.

- Отсечение: Если текущее количество размещенных квадратов `count` становится больше или равно уже найденному лучшему значению `min_squares[0]`, ветка поиска немедленно прекращается.

- Ограничение `max_fit`: Функция `get_max_size` анализирует текущую свободную клетку и сразу возвращает максимально возможный размер квадрата. Это исключает перебор размеров, которые физически не поместятся на поле.

- Глобальное ограничение: Введен запрет на установку квадратов, размер которых равен стороне поля, согласно правилу варианта 4р, что исключает постановку "недопустимых" фигур.

4. Описание используемых функций

1. `is_safe(row, column, size, board, n, m)`

Функция проверки возможности размещения квадрата. Проверяет, не выходит ли квадрат за границы прямоугольного поля и не перекрывает ли он уже занятые клетки.

- Аргументы: `row`, `column` — координаты левого верхнего угла, `size` — сторона квадрата, `board` — матрица поля, `n`, `m` — размеры поля.

- Возвращает: `True`, если размещение возможно, иначе `False`.

2. `add_square(row, column, size, val, board)`

Функция управления состоянием поля. Используется для размещения квадрата (установка 1) и для удаления (установка 0 при откате).

- Аргументы: `row`, `column` — координаты, `size` — сторона квадрата, `val` — значение (1 или 0), `board` — матрица поля.

- Результат: Матрица `board` модифицируется напрямую.

3. `find_empty(n, m, board)`

Функция поиска первой свободной клетки. Последовательно просматривает все ячейки матрицы `board`, начиная с нулевой строки и нулевого столбца.

- Возвращает: Кортеж `(row, column)` с координатами первой найденной свободной ячейки или `False`, если свободных ячеек не осталось.

4. `get_max_size(row, column, board, n, m)`

Функция определения максимально допустимого размера квадрата. Она итеративно увеличивает сторону квадрата до тех пор, пока он помещается на поле и не встречает препятствия (уже установленные квадраты).

- Аргументы: `row`, `column` — координаты текущей свободной клетки, `board` — матрица поля.
- Возвращает: `size` — максимально возможное целое число, представляющее сторону квадрата.

5. `backtrack(count, current_list, n, m, board, min_squares, count_variants, best_list)`

Основная рекурсивная функция поиска. Реализует метод перебора с возвратом. Находит первую свободную клетку, определяет возможный диапазон размеров квадратов и последовательно пробует ставить их, уходя в рекурсию. При достижении полного покрытия поля сравнивает полученный результат с лучшим найденным, обновляя рекорд или счетчик вариантов.

6. Описание рекурсивной функции.

В работе выполнена функция `backtrack`.

Сигнатура: `backtrack(count, current_list, n, m, board, min_squares, count_variants, best_list)`

Назначение: Поиск оптимального покрытия прямоугольника и подсчет числа комбинаций.

Описание:

1. Проверка условия отсечения (`count >= min_squares[0]`).
2. Поиск первой свободной клетки через `find_empty()`.
3. При отсутствии свободных клеток — обновление глобального рекорда или счетчика вариантов.
4. Определение максимально возможного размера квадрата (`get_max_size`).
5. Циклический перебор размеров: установка квадрата, рекурсивный вызов, удаление квадрата.

7. Оценка сложности алгоритма

Временная сложность:

$O(b^h)$, где b — среднее число допустимых размеров квадратов, h — глубина рекурсии. Алгоритм имеет экспоненциальную сложность, типичную для задач полного перебора, однако оптимизации позволяют находить решения для заданных ограничений ($N, M \leq 20$) за приемлемое время.

Пространственная сложность:

$O(N \times M)$ для хранения поля и стека рекурсии, глубина которого ограничена общим числом квадратов.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования

№	Входные данные	Выходные данные
1	5 6	Минимальное число квадратов: 5 1 1 3 1 4 3 4 1 2 4 3 2 4 5 2 Всего найдено вариантов оптимального покрытия: 2
2	7 9	Минимальное число квадратов: 7 1 1 5 1 6 4 5 6 1 5 7 3 6 1 2 6 3 2 6 5 2 Всего найдено вариантов оптимального покрытия: 4
3	15 15	Итоговые результаты: Минимальное число квадратов: 6 1 1 10 1 11 5 6 11 5 11 1 5 11 6 5 11 11 5 Всего найдено вариантов оптимального покрытия: 4

Выводы

В ходе лабораторной работы был реализован алгоритм поиска с возвратом для решения задачи покрытия прямоугольной области минимальным количеством квадратов. Внедрение методов отсечения ветвей, основанных на хранении текущего глобального минимума, позволило значительно сократить пространство перебора и обеспечить высокую производительность алгоритма для полей размером до 20×20 .

Модификация алгоритма в соответствии с вариантом 4р обеспечила корректную обработку прямоугольных полей и позволила реализовать механизм подсчета всех вариантов оптимального покрытия. Использование вспомогательных функций анализа пространства для установки квадратов исключило избыточные проверки.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.py

```
def add_square(row, column, size, val, board):
    for x in range(row, row + size):
        for y in range(column, column + size):
            board[x][y] = val

def find_empty(n, m, board):
    for row in range(n):
        for column in range(m):
            if board[row][column] == 0:
                return row, column
    return False

def get_max_size(row, column, board, n, m):
    size = 1
    while row + size < n and column + size < m:
        conflict = False
        for i in range(size + 1):
            if board[row + size][column + i] == 1 or board[row + i][column
+ size] == 1:
                conflict = True
                break
        if conflict:
            break
        size += 1
    return size

def backtrack(count, current_list, n, m, board, min_squares,
count_variants, best_list):
    if count > min_squares[0]:
        return

    res = find_empty(n, m, board)

    if res is False:
```

```

        if count < min_squares[0]:
            print(f">>> [НОВЫЙ РЕКОРД] Найдена сборка из {count}
квадратов!")
            min_squares[0] = count
            count_variants[0] = 1
            best_list[0] = list(current_list)
        elif count == min_squares[0]:
            if count_variants[0] < 5:
                print(f"--- [АЛЬТЕРНАТИВА] Найдена еще одна сборка из
{count} квадратов.")
                count_variants[0] += 1
            return

    row, column = res
    max_fit = get_max_size(row, column, board, n, m)

    max_fit = min(max_fit, n - 1, m - 1)

    for size in range(max_fit, 0, -1):
        add_square(row, column, size, 1, board)
        current_list.append((row, column, size))

        backtrack(count + 1, current_list, n, m, board, min_squares,
count_variants, best_list)

        current_list.pop()
        add_square(row, column, size, 0, board)

print("Введите размеры поля N M:")
n, m = map(int, input().split())

board = [[0] * m for _ in range(n)]
min_squares = [n * m]
count_variants = [0]
best_list = [[]]

print("\n--- Старт поиска ---")
backtrack(0, [], n, m, board, min_squares, count_variants, best_list)

```

```
print("--- Поиск завершен ---\n")

print("Итоговые результаты:")
print(f"Минимальное число квадратов: {min_squares[0]}")
for r, c, size in best_list[0]:
    print(f"{r + 1} {c + 1} {size}")

print(f"\nВсего      найдено      вариантов      оптимального      покрытия:
{count_variants[0]}")
```